IEEE Systems Engineering Processes and Extreme Programming

6.4.2 Stakeholder Needs and Requirements Definition process

1. Prepare for Stakeholder Needs and Requirements Definition

a. Identify the stakeholders who have an interest in the software system throughout its lifecycle.

- **Process outcome:** a
- **XP implementation:** Early on, the team lists everyone the software touches: end users, the customer, the people who will maintain it, whoever operates it, and anyone else affected. The on-site customer speaks for most of them and pulls in the rest when a story needs their input.

b. Define the stakeholder needs and requirements definition strategy.

- **Process outcome:** d
- **XP implementation:** We find out what people need by talking to the customer often. Each need becomes a user story, backed by examples and acceptance tests. Stories get another look at every release and iteration planning session, so new feedback can change them.

c. Identify and plan for the necessary enabling systems or services needed to support stakeholder needs and requirements definition.

- **Process outcome:** h
- **XP implementation:** The team works out what it needs to capture stories, stay in touch with stakeholders, and run acceptance tests. Typically that's a shared story board, a room or video link for conversations, and an automated test framework.

d. Obtain or acquire access to the enabling systems or services to be used.

- **Process outcome:** h
- **XP implementation:** Someone sets up the story board and the test environment, then gives the customer and developers accounts. Before relying on them, the team confirms that stakeholders can actually take part and that acceptance tests run.

2. Define stakeholder needs

a. Define context of use within the concept of operations and the preliminary life cycle concepts.

- **Process outcome:** b
- **XP implementation:** The customer explains to the team who will use the software, what they're trying to get done, and where. That goes into stories and examples. Small releases show what was wrong or missing, and the context gets corrected from there.

b. Identify stakeholder needs.

- **Process outcome:** d
- **XP implementation:** Needs come from talking with users and other stakeholders, watching them work, and showing them working software. Each one is written as a story, with examples to pin down what it means.

c. Prioritize and down-select needs.

- **Process outcome:** e
- **XP implementation:** In the planning game, the customer ranks stories by business value. Developers estimate the effort and flag technical risk. The team then picks the highest-value stories it can realistically finish in the next release or iteration, and the rest wait.

d. Define the stakeholder needs and rationale.

- **Process outcome:** d
- **XP implementation:** A story should say what the stakeholder wants and why. The customer and developers rework each one until both are clear, and the reason stays attached to the story. Details get settled in conversation, with examples, before anyone writes code.

3. Develop the operational concept and other life cycle concepts

a. Define a representative set of scenarios to identify the required capabilities that correspond to anticipated operational and other life cycle concepts.

- **Process outcome:** b
- **XP implementation:** Story examples and acceptance-test scenarios double as the representative scenarios. They cover normal use as well as the important failures and how the user recovers. When a scenario exposes something the system can't do yet, it becomes a new story or a new acceptance criterion.

b. Identify the factors affecting interactions between users and the system.

- **Process outcome:** b
- **XP implementation:** The team talks through who the users are: what they can physically and mentally do, what they've been trained on, what equipment and workplace they have, and what happens in unusual or emergency conditions. Accessibility, usability, and operational needs that come out of this go into stories and acceptance examples. Representative users then check them.

4. Transform stakeholder needs into stakeholder requirements

a. Identify the constraints on a system solution.

- **Process outcome:** c
- **XP implementation:** Constraints like required platforms, interfaces, deadlines, budgets, and contractual terms are recorded next to the stories they affect. If a constraint is unclear, a short technical spike can settle it before the team commits to an approach.

b. Identify the stakeholder requirements and functions that relate to critical quality characteristics, such as assurance, safety, security, environment, or health.

- **Process outcome:** e
- **XP implementation:** Quality needs get their own stories or explicit acceptance criteria, with measurable targets where possible, such as a security rule or a performance limit. The team plans the work and writes specific checks for them. Ordinary functional tests won't cover these by accident.

c. Define stakeholder requirements, consistent with life cycle concepts, scenarios, interactions, constraints, and critical quality characteristics.

- **Process outcome:** e
- **XP implementation:** Prioritized needs become clear stories with acceptance criteria that reflect the scenarios, constraints, and quality targets already agreed. The customer and developers keep adjusting them as they learn more.

5. Analyze stakeholder requirements

a. Analyze the complete set of stakeholder requirements.

- **Process outcome:** e
- **XP implementation:** During planning and refinement, the customer and developers read through the stories and criteria together, looking for gaps, vague wording, conflicts, anything infeasible, and anything that can't be tested. Problem stories get split or rewritten. If the team lacks information, it runs a short spike.

b. Define critical performance measures that enable the assessment of technical achievement.

- **Process outcome:** f
- **XP implementation:** For the qualities that matter most, the customer and team agree on a number, for example a response time under a given load. Acceptance or performance tests check it, and the team looks at the results as the system grows.

c. Feed back the analyzed requirements to applicable stakeholders to validate that their needs and expectations have been adequately captured and expressed.

- **Process outcome:** g
- **XP implementation:** The customer takes the refined stories and acceptance examples back to the relevant stakeholders. Regular demos of working software give them another chance to say "that's not what I meant."

d. Resolve stakeholder requirements issues.

- **Process outcome:** g
- **XP implementation:** When stories are unclear or stakeholders disagree, the customer gets the affected people and the developers in one conversation. The story and its criteria are updated, priorities are revisited if needed, and the people involved confirm the result.

6. Manage the stakeholder needs and requirements definition

a. Obtain explicit agreement with designated stakeholders on the stakeholder requirements.

- **Process outcome:** g
- **XP implementation:** Before the team builds on a set of stories, the customer goes over them with the designated stakeholders and records their agreement. As stories are finished, the customer accepts them against the agreed criteria. If a requirement changes, the agreement is revisited.

b. Maintain traceability of stakeholder needs and requirements.

- **Process outcome:** i
- **XP implementation:** Each story links back to who asked for it and why, forward to its acceptance tests, and to the related code or issue records. When a story is split, changed, or replaced, the links are updated so the reasoning isn't lost.

c. Provide key artifacts and information items that have been selected for baselines.

- **Process outcome:** i
- **XP implementation:** The baseline is the agreed story set, its acceptance criteria, the stakeholder links, and the current release or iteration plan. It lives in a shared, version-controlled place and is updated only when stakeholders agree to a change.

6.4.4 Architecture Definition process

Listed Outcomes: a) Identified stakeholder concerns are addressed by the architecture.

b) Architecture viewpoints are developed.

c) Context, boundaries, and external interfaces of the system are defined.

d) Architecture views and models of the system are developed.

e) Concepts, properties, characteristics, behaviors, functions, or constraints that are architecture decisions of the system are allocated to architectural entities.

f) System elements and their interfaces are identified.

g) Architecture candidates are assessed.

h) An architectural basis for processes throughout the life cycle is achieved.

i) Alignment of the architecture with requirements and design characteristics is achieved.

j) Any enabling systems or services needed for the architecture definition are available.

k) Traceability of architecture elements to stakeholder and system/software requirements is developed.

a. Prepare for architecture definition

1) Review pertintent information and identify key drivers of the architecture.

a. a

b. The team and customer write user stories, and the team runs short spikes on risky technical questions. The key drivers surface in release planning, where the customer sets priorities and the team states the quality constraints (performance, security, platform).

2) Identify stakeholder concerns.

a. a

b. The on-site customer is the main source of concerns, and the planning game turns them into user stories. Other stakeholders' concerns come in through the customer or through frequent iteration demos.

3) Define the Architecture Definition roadmap, approach, and strategy.

a. h

b. The release plan and iteration plans act as the roadmap. The strategy is evolutionary: build the simplest architecture that works, guided by the system metaphor, and refine it every iteration instead of designing it all up front.

4) Define architecture evaluation criteria based on stakeholder concerns and key requirements.

a. g

b. The customer writes acceptance criteria and the team adds measurable targets for non-functional needs, such as response time. Both are captured as automated acceptance or performance tests that any architectural choice must pass.

5) Identify and plan for the necessary enabling systems or services needed to support the Architecture Definition process.

a. j

b. The team plans the shared tooling in an early iteration (sometimes called iteration zero): a version control repository, a continuous integration server, test frameworks, an issue or story tracker, and a shared workspace with a whiteboard.

6) Obtain or acquire access to the enabling systems or services to be used.

a. j

b. The team sets up the repository, CI build, test harness and pairing workstations before feature work starts. Everyone gets access, and collective ownership means anyone can use the tools.

b. Develop architecture viewpoints.

1) Select, adapt, or develop viewpoints and model kinds based on stakeholder concerns.

a. b

b. XP keeps viewpoints minimal. The system metaphor gives a shared high-level view, and CRC cards or quick sketches give a structural view. Acceptance tests give the customer's behavioral view.

2) Establish or identify potential architecture framework(s) to be used in developing models and views.

a. b

b. XP does not use a formal architecture framework. The team agrees on the metaphor, coding standards, chosen technology stack and common design patterns, and that agreement serves as the lightweight framework.

3) Capture rationale for selection of framework(s), viewpoints and model kinds.

a. b

b. Rationale is recorded informally: notes on story cards, a team wiki, commit messages and retrospective notes. The tests and code are treated as the main documentation, so little separate paperwork is produced.

4) Select or develop supporting modeling techniques and tools.

a. b

b. The team uses whiteboards, CRC cards, informal UML sketches, an IDE with refactoring support, and the automated test framework.

c. Develop models and views of candidate architectures.

1) Define the software system context and boundaries in terms of interfaces and interactions with external entities.

a. c

b. User stories and acceptance tests describe how the system interacts with users and external systems. For external dependencies, the team writes spikes and uses mocks or stubs so the boundary is exercised by tests early.

2) Identify architectural entities and relationships between entities that address key stakeholder concerns and critical software system requirements.

a. d

b. The team finds the main classes or components and their collaborations with CRC cards and the system metaphor. Pairs and the whole team discuss these at the whiteboard.

3) Allocate concepts, properties, characterisics, behaviors, functions, or constraints that are significant to architecture decisions of the software system to architectural entities.

a. e

b. Responsibilities are assigned to classes or components through CRC sessions and simple design. As stories are implemented, the team refactors to move responsibilities to where they fit best.

4) Select, adapt, or develop models of the candidate architectures of the software system.

a. d

b. The team sketches alternative designs and tests the risky ones with spikes. These are throwaway prototypes that answer a specific technical question quickly.

5) Compose views from the models in accordance with identified viewpoints to express how the architecture addresses stakeholder concerns and meets stakeholder and system/software requirements.

a. d

b. The views are whiteboard diagrams, the metaphor and the passing test suite. Diagrams are produced when they are needed and kept lightweight.

6) Harmonize the architecture models and views with each other.

a. d

b. Collective code ownership, continuous refactoring and daily team communication keep the sketches, tests and code consistent. When the design drifts, the team fixes it through refactoring and a quick design discussion.

d. Relate the architecture to design.

1) Identify software system elements that relate to architectural entities and the nature of these relationships.

a. f

b. CRC cards and metaphor names map directly onto classes, modules and packages in the code. The code structure itself shows which elements implement which entity.

2) Define the interfaces and interactions among the software system elements and external entities.

a. f

b. Test-driven development defines interfaces by writing tests first. Integration tests and mocks pin down interactions between elements and with external systems.

3) Partition, align and allocate requirements to architecture entities and system elements.

a. i

b. Stories are split during the planning game and assigned to iterations and pairs. Each story’s acceptance tests are tied to the components that implement it.

4) Map software system elements and architecture entities to design characteristics.

a. i

b. Simple design rules (passes all tests, no duplication, expresses intent, minimal elements) and coding standards set the design characteristics. Refactoring brings the elements into line with them.

5) Define principles for the software system design and evolution

a. h

b. XP’s principles are simplicity, YAGNI, once and only once, continuous refactoring, small releases and coding standards. They guide how the design evolves.

e. Assess architecture candidates.

1) Assess each candidate architecture against constraints and requirements.

a. g

b. Spikes, automated acceptance tests and performance or load tests show whether a candidate meets constraints and requirements. The result is evidence from working code.

2) Assess each candidate architecture against stakeholder concerns using evaluation criteria.

a. g

b. The customer reviews working software in iteration demos and checks it against the agreed acceptance criteria. Feedback shapes the next planning game.

3) Select the preferred architecture(s) and capture key decisions and rationale.

a. g

b. The whole team decides, usually at planning, in a stand-up or at the whiteboard, and picks the simplest option that satisfies the tests. The decision is noted on cards, in the wiki or in commit messages.

4) Establish the architecture baseline of the selected architecture.

a. h

b. The baseline is a tagged version in source control with a green CI build, passing tests and a deployed small release.

f. Manage the selected architecture.

1) Formalize the architecture governance approach and specificy governance-related roles and responsibilities, accountabilities, and authorities related to the design, quality, security, and safety.

a. h

b. Governance is team-based. The coach keeps the team on the practices, the customer has authority over priorities and acceptance, and the whole team shares responsibility for design quality through collective ownership, pair programming and coding standards. Security and safety needs become stories and tests.

2) Obtain explicit acceptance of the architecture by stakeholders.

a. a

b. The customer accepts the work by running the acceptance tests and reviewing each iteration's working software. Acceptance happens continuously, not in one sign-off.

3) Maintain concordance and completeness of the architectural entities and characteristics.

a. i

b. Continuous integration, the automated test suite and ongoing refactoring keep the architecture consistent and complete as the code changes.

4) Organize, assess and control evolution of the architecture mdoels and views to help ensure that the architectural intent is met and the architectural vision and key concepts are correctly implemented.

a. h

b. The team controls evolution through small releases, refactoring, pair programming, code reviews and retrospectives. The metaphor is revisited and updated so the shared vision stays accurate.

5) Maintain the architecture definition and evaluation strategy.

a. h

b. Retrospectives and iteration or release planning are where the team revises its architectural approach, tests and criteria as it learns.

6) Maintain traceability of the architecture.

a. k

b. Traceability runs from story card to acceptance test, to code and to commit or issue ID. Test names and the version control history show which requirement drove each element.

7) Provide key artifacts and information items that have been selected for baselines.

a. h

b. The key artifacts are the working software, the automated test suite, the tagged releases and the minimal documentation the team chose to keep (such as the metaphor description and the story and test records).

6.4.5 Design Definition process

Listed Outcomes

1. Design characteristics of each system element are defined (a)
2. System/software requirements are allocated to system elements (b)

3. Design enablers necessary for design definition are selected or defined (c)
4. Interfaces between system elements composing the system are defined or refined (d)
5. Design alternatives for system elements are assessed (e)
6. Design artifacts are developed (f)
7. Any enabling system or services needed for design definition are available. (g)
8. Traceability of the design characteristics to the architectural entities of the system architecture is established. (h)

Activity and Task responses

1. Prepare for software system design definition

a. Define the design definition strategy, consistent with the selected life cycle model and anticipated design artifacts

a. c.

b. The team agrees on an incremental, evolutionary design strategy in release planning. The life cycle is short iterations and small releases. According to the book: "The goal is to have deployable software at the end of the week which everyone can celebrate as progress."

b. Select and prioritize design principles and design characteristics

a. a.

b. Use the Simple Design rules: pass all tests, no duplication, express intent, minimal elements. Add coding standards and the system metaphor for naming and shared vocabulary. The customer prioritizes characteristics through user stories. According to the book: "Have the customers pick a week’s worth of stories to implement this week."

c. Identify and plan for the necessary enabling systems or services needed to support design definition

a. g

b. Plan the team’s tooling in the first iteration, or in an “iteration zero”. This covers the version control repository, the continuous integration server, the test framework, the shared workspace, whiteboards, and index cards. According to the book: "Automatically build the whole system and run all of the tests in ten minutes."

d. Obtain or acquire access to the enabling systems or services used

a. g

b. Set up the repository, CI/build server, automated test harness, and IDE in the first iteration. Give every pair access and use a single integration machine. Collective code ownership means everyone has access to

everything. According to the book: "Automatically build the whole system and run all of the tests in ten minutes."

2. Establish designs related to each software system element

a. Transform architectural and design characteristics into the design of software system elements

a. a and b

b. Turn user stories into engineering tasks in the iteration planning meeting. Allocate tasks to classes and components using quick design sessions. Write acceptance tests and unit tests first (TDD) so the test is the design specification of each element. According to the book: "Start the week by writing automated tests that will run when the stories are completed."

b. Define and prepare or obtain the necessary design enablers

a. c

b. Set the coding standard and the system metaphor. Prepare the unit test framework, refactoring tools, design patterns the team agrees on, and the CI pipeline. Pair programming spreads design knowledge as an enabler. According to the book: "Pair programming is a dialog between two people simultaneously programming (and analyzing and designing and testing) and trying to program better."

c. Examine design alternatives and feasibility of implementation

a. e

b. Use spike solutions, which are small throwaway prototypes that test risky technical options. Pair programming gives immediate peer review, and the “simplest thing that could possibly work” rule picks among alternatives. According to the book: "You can even prototype alone and still respect pairing...When you’re done exploring, bring the resulting idea, not the code, back to the team."

d. Refine or define the interfaces among the software system elements and with external entities

a. d

b. Define interfaces through tests. Unit tests specify the contracts between classes, and acceptance tests cover external interfaces. Continuous integration exposes interface mismatches early, and refactoring refines the interfaces each iteration. The client clarifies external interfaces. According to the book: "The integration step is unpredictable, but can easily take more time than the original programming."

e. Establish the design artifacts

a. f

b. The artifacts are the working code, the automated unit and acceptance tests, story cards, and the system metaphor. Refactoring keeps the code self-documenting and the design clean. According to the book: "Start the week by writing automated tests that will run when the stories are completed...Another implementation of the informative workspace is big, visible charts."

3. Assess alternatives for obtaining software system elements

a. Determine technologies required for each element composing the software system

a. e

b. Identify the technology needs while estimating stories. Technology risk gets a spike. The team picks the technology that supports the current stories, and no more. According to the book: "As soon as a story is written, try to estimate the development effort necessary to implement it."

b. Identify candidate alternatives for the software system elements

a. e

b. Brainstorm in the whole-team design sessions. Candidates include build vs. reuse, existing libraries, and frameworks. Spikes make the candidates concrete. According to the book: "Brainstorm refinements to the system...The weekly heartbeat also gives you a convenient, frequent, and predictable platform for team and individual experiments."

c. Assess each candidate alternative against criteria developed from expected design characterisitics and element requirements to determine suitability for the intended application

a. e

b. The criteria are the simple design rules, the acceptance tests, and the story estimates. Run the spike against the acceptance criteria, compare the results, and judge simplicity and testability. According to the book: "Estimation gives the business and technical perspectives a chance to interact, which creates value early, when an idea has the most potential."

d. Choose the preferred alternatives among candidate design solutions for the software system elements

a. e

b. The team decides by consensus and picks the simplest alternative that satisfies the stories. The customer weighs in where cost or scope is affected. The decision is reversible, because refactoring lets the team change it later. According to the book: "You can always add more stories later and deliver more than you promised"

4. Manage the design

a. Capture the design and rationale

a. f

b. Rationale lives in the code, test names, commit messages, story cards, and wiki/whiteboard photos. Tests act as executable documentation. Brief decision notes are written only when needed. According to the book: ""“Give stories short names in addition to a short prose or graphical description...The results will be more widely understood, benefitting the project as a whole"

b. Establish traceability between the detailed design elements, the system/software requirements, and the architechtural entities of the software system architechture

a. h

b. Each user story maps to acceptance tests, tasks, and commits (story IDs in commit messages). The system metaphor and consistent naming tie code to architecture. Traceability is lightweight and comes from the test suite and version control history. According to the book: "Start the week by writing automated tests that will run when the stories are completed..If you need to report progress to other parts of the organization in a familiar format, translate the cards into that format periodically."

c. Determine the status of the software system and element design

a. f

b. Status is visible through passing automated tests, continuous integration builds, the task board, the iteration burn-down or velocity tracking, and the daily stand-up. Frequent refactoring and code review through pairing show design health. According to the book: "An interested observer should be able to walk into the team space and get a general idea of how the project is going in fifteen seconds."

d. Provide key artifacts and information items that have been selected for baselines.

a. f

b. Each small release is tagged in version control as the baseline. The baseline holds the tested code, the acceptance and unit test suites, and the release notes. The customer accepts it at the end of the iteration or release. According to the book: "The goal is to have deployable software at the end of the week which everyone can celebrate as progress."